

★★ 6. Direct Field Access

Overview

This documentation describes the implementation of direct field access in methods, allowing fields to be referenced by name without explicit `this.` qualification. The implementation includes proper handling of name shadowing and inheritance.

Feature Requirements

- Direct field reference within methods using just the field name
- Support for accessing superclass fields
- Proper shadowing rules where method parameters and local variables take precedence
- Maintenance of type safety

Implementation Details

1. Field Reference Expression

Added new expression form for field references:

```
| fieldI(field_name :: Symbol)
```

2. Type Checking Implementation

The type checker handles field references with proper shadowing rules:

```
| fieldI(field_name):  
  match this_type  
  | objT(class_name):  
    def maybe_env_type = try:  
      find(env, field_name)  
    ~catch:
```

```

        find_field_in_tree(field_name, class_name, t_classes)
    maybe_env_type
    | nullT():
        type_error(exp, "cannot access field of null")
    | ~else:
        type_error(exp, "can't access fields outside of classes")

```

Key features:

1. Name Resolution Priority:

- First checks local environment (method parameters and local variables)
- Falls back to field lookup in class hierarchy
- Ensures proper shadowing semantics

2. Inheritance Support:

- Uses `find_field_in_tree` for superclass field lookup
- Maintains type information through inheritance chain

3. Context Validation:

- Ensures field access occurs within class context
- Prevents access on null
- Maintains type safety

3. Translation Implementation

Field references are translated to explicit `this` accesses:

```

| fieldI(field_name):
    getE(thisE(), field_name)

```

This translation ensures:

- Consistent runtime behavior

- Proper field access semantics
- Compatibility with existing object model

4. Parser Implementation

Extended parser to handle field references:

```
| syntax_is_symbol(s):
  let name = syntax_to_symbol(s):
    cond
    | name == #'arg: argI()
    | name == #'this: thisI()
    | ~else: fieldI(name)
```

Features:

- Handles bare identifiers as potential field references
- Preserves special handling of 'arg' and 'this'
- Integrates with existing expression parsing

Testing

Comprehensive test cases demonstrate correct behavior:

```
check: parse_method('method getX(arg): x')
  ~is values(#'getX, fieldI(#'x))

check: typecheck_exp(fieldI(#'x),
  [field_t_class],
  objT(#'A),
  intT(),
  [])
  ~is intT()
```

Test coverage includes:

1. Basic field access

2. Shadowing behavior:

- By method parameters
- By local variables

3. Inheritance:

- Superclass field access
- Multiple inheritance levels

4. Error cases:

- Non-existent fields
- Access outside class context
- Null receiver

Design Choices and Rationale

1. Name Resolution Strategy

- Clear priority order for name resolution
- Consistent with traditional OOP scoping rules
- Predictable shadowing behavior

2. Translation Approach

- Translation to explicit `this` access
- Reuse of existing runtime mechanisms
- Simplified implementation without runtime changes

3. Inheritance Support

- Transparent access to inherited fields
- Consistent with object-oriented principles
- Maintained type safety through inheritance chain

Type Safety

The implementation maintains type safety through:

1. Static Verification

- Field existence checking
- Type preservation through inheritance
- Context validation

2. Shadowing Safety

- Proper precedence rules
- Clear resolution strategy
- Type consistency in all contexts

3. Translation Correctness

- Type-preserving transformation
- Runtime consistency
- Safe field access

Conclusion

The implementation successfully adds direct field access to the language while maintaining type safety and providing clear semantics for name resolution. The design choices create a natural and intuitive field access mechanism that integrates well with the existing type system and object model.